SYS3 LAB ACTIVITY 3 : Instruction Reordering

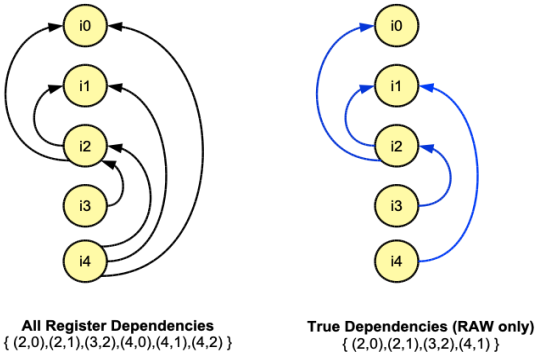
Weeks 9 (typical study period)

Introduction:

Instruction reordering is a method employed statically during compilation, or dynamically in a highly complex processor. It is best understood by looking at the static approach, which acts upon the source machine code sequence.

Consider the example below, where a code sequence has a **register dependency graph**, such that any instruction pair (x,y) represents a potential[1] dependency of instruction x upon instruction y. For instance, instruction i3 depends upon instruction i1 and i2 due to its input dependencies of R5 and R6.

We can draw an initial graph of the instructions in order, as show on the left hand of the **register dependency graph** representation.

Initial Program		**Renamed Program**
i0 LDI R5, 12		i0 LDI R5, 12
i1 LDI R6, 10		i1 LDI R6, 10
i2 IADD R5,R6,R7		i2 IADD R5,R6,R7
i3 IMUL R7,R7,R7		i3 IMUL R7,R7,R17
i4 IADD R6,R2,R5		i4 IADD R6,R2,R15

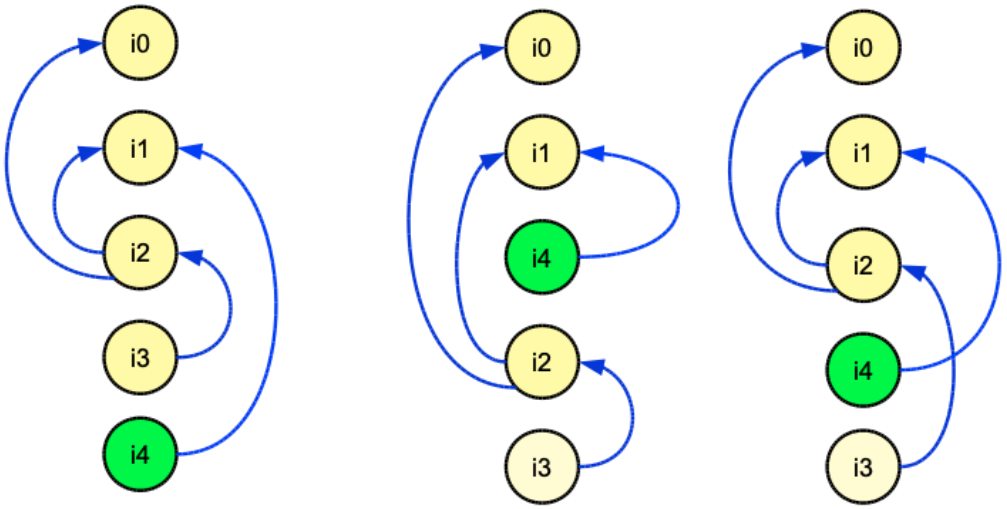
However Some of the dependencies are due to WAW and WAR hazards - they are due to the *possibility* of Write and Read events getting out of order, and can be dealt with by using renaming. These are known as *anti-dependencies* (WAR), and *output dependencies* (WAW). However, this still leaves RAW hazards (also know as a *true dependences), still present,* as can be seen on the right hand side of the diagram.

[1] Potential, because we cannot really know unless we evaluate the timing of events in the pipeline. The safest approach is to assume that a potential hazard is an actual hazard until we can decide otherwise. Not doing so may result in incorrect code generation.

True dependencies occur because actual data from one instruction is needed by a later instruction, and they are therefore data dependent. As we found out in previous exercises (weeks 6/7), RAW cannot be resolved simply by using register renaming.

So, we can eliminate WAW and WAW hazards at this point, because we know renaming will fix that problem. But we would ideally like to fix the RAW problem too. Our basic solution would be to insert stalls, as we did in earlier lab exercises, but this creates wasted cycles in the pipeline. It would be far better to create the equivalent delays by inserting other useful instructions into the pipeline to separate the Write and Read events in the RAW dependency as much as possible to eliminate the problem. This is effectively a case of instruction reordering.

Consider the second graph below:-



Original Code
{ 0,1,2,3,4 }

Option 1 of several
{ 0,1,4,2,3 }

Option 2 of several
{ 0,1,2,4,3 }

We can see that instruction 4 can move to earlier slots because it is only dependent on instruction 1. So there are at least three possible instruction orders we can envisage, each with the same overall effect. In fact, Instruction 0 can also move around in some positions, so there are more than 3 possibilities. Some of these are only possible after renaming has been applied.

You might also see that Option1 is possibly no better than the original code, but Option2 is possibly an improvement. Why is that ? - consider what was explained on the previous page and think about the two options shown.

What we can see is that if we take the original program, and change the order of the instructions, we ***might*** reduce the occurrence of a RAW hazard, or at least reduce the number of stalls required to deal with them. This could result in improved efficiency of the code. To some extent, instruction reordering is a guess, unless the algorithm is very sophisticated, we will not know the true pipeline behaviour unless the code is executed or emulated. This is possible, but beyond the scope of this module.

10.1 Initial Preparation

In previous lab sessions, we have achieved a number of optimisation tasks, dealing with a range of hazards:-

1. Structural Hazards
2. Memory Hazards
3. Register Hazards

Solutions to all of these problems are combined in the test program Week9.cpp to give what appears to be a near-complete solution to the problem of hazards in pipelines using dynamic pipeline scheduling techniques. We will apply these techniques as a starting point and then investigate some reordering ideas.

CPU Configuration

- Issue Width = 1
- Register Read Ports = 3
- Register Write ports = 1
- Integer ALU = 3
- Float ALU = 1
- Shift/Binary Unit = 1
- CacheMode = 0 (unified cache)

Test Cases

- **test9a.txt**
- **test9b.txt**

i. Build week9a.cpp and test it with a few of the testcases to make sure it works. You should see that all hazards are eliminated.

ii. Test program week9.cpp includes the function DetectDependencies(), and this can be used to create lists of (x,y) instruction dependency pairs. Use this to sketch dependency graphs for test case **test9a.txt** and **test9b.txt**.

iii. Compare your dependency graphs to the ones presented on the previous page, (these are based upon the same test cases). You will be able to see that they are the same.

more overleaf …

10.1 Manual Reordering Experiments

In this exercise we will explore some reordering cases, using test case **test9a.txt** , the same one presented in the diagrams, and also **test9b.txt**, which is the same program after manually applying renaming.

CPU Configuration

- Issue Width = 1
- Register Read Ports = 2
- Register Write ports = 1
- Integer ALU = 1
- Float ALU = 1
- Shift/Binary Unit = 1
- CacheMode = 0 (unified cache)

Test Cases

- **test9b.txt**
- **test9a.txt**

Tasks

i. Make a list of the possible instruction reorderings for the two test cases and the code with register renaming, where the original code has the ordering {1,2,3,4,5}.

In theory there are 120 (n!) reorderings for n=5 instructions, however most of these will not be valid. Rather than trying to list these exhaustively, simply try to identify as many valid cases as you can.

ii. For each of the test cases **test9a.txt** and **test9b.txt.**, create several new files with different valid reorderings. You can label these as **test9a1 test9a2**, etc.

iii. Run the Combined optimisation program **week9.cpp** on each of the test cases including the originals and complete the table below.

test case & { order }	**Serial Cycles**	**Pipelined cycles**	**Speedup**
test9a.txt {1,2,3,4,5}			
test9a1.txt { xxxxx }			
test9a2.txt { xxxxx }			
…			
etc			

You should be able to see that some cases work better than others in terms of requiring less cycles. You may also notice that the manually renamed program in **test9b.txt** allows for more cases to be generated. It may be that the best cases

can only be achieved by using renaming, or it may not be the case. However, what we can say is that in general, Renaming will provide more possible ways to reorder a given piece of code, and therefore make more opportunities to get a better result.

10.2 Reordering Algorithm experiments

Implementing instruction reordering in a full solution is not an easy task, and there is some variation as to where it may be implemented (in the compiler, statically, or dynamically in the CPU for example).

We can make some inroads into exploring a solution in this practical activity however. First of all consider how we would generate all of the possible valid permutations, which we attempted to do manually in 10.1.

A possible partial algorithm might be as follows:-

1. Identify a list or grid[2] of all possible register dependencies.
2. Generate every possible instruction ordering of the n! permutations.
3. Test each permutation against the dependency list to eliminate invalid permutations.

For example, if testcase9b.txt has a list of dependencies { (2,0),(2,1),(3,2),(4,1) } then consider these two orderings:-

{0,1,2,3,4}	Valid because for every instruction x in an (x,y) pair, instruction y occurs earlier in the list.
{0,1,**3,2**,4}	Not valid, because for instruction **x=3** there is an (x,y) dependency pair (3,2) and instruction **y=2** appears AFTER instruction x=3, which breaks the dependency.

So we can devise a function that creates a list of pairs, another function that generates all possible permutations, and finally modify this so it checks that the list of pairs is not violated, then prints out each permutation with a valid/invalid status (or just prints the valid pairs) .

Task:

Using the same configurations for CPU and test cases, try to develop a reordering algorithm that generates an accurate list of valid reorderings, and compare these to your lists from 10.1.

You may find useful permutation solutions on the web, for example:-

https://stackoverflow.com/questions/70758782/printing-all-permutations-of-n-numbers

[2] A simple way may be to create a 2D array containing all X,Y pairings as 0/1 values.

Advanced (optional) : once you can generate valid reorderings, you may want to attempt to develop a complete solution. (Complete part 10.2 first however).

The key thing here is that for every valid reordering, we must generate a test program, run it through the emulator, and determine how many clock cycles it requires (the cost factor). The permutation with the lowest cycle count and no hazards would be the best choice.

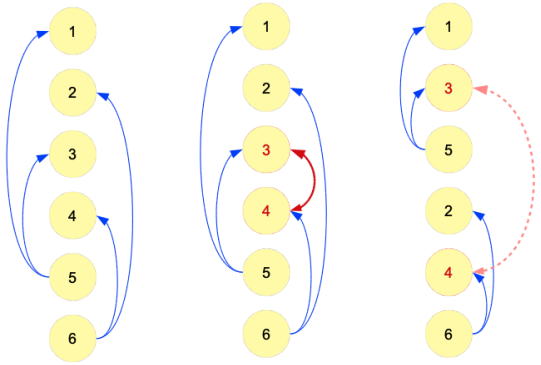
There are ways to achieve this outcome but some guidance will likely be needed to get to a reasonably straightforward implementation.

Discuss this with the tutor for some ideas before attempting this !.

10.2 Reordering - non register hazards

So far we have considered reordering only to resolve the problem of RAW hazards, which cannot be fixed using other techniques. The principle is that where there is a conflict between two entities (in that case a Read and a write to the same register), we try to separate those two entities in time by inserting other useful (but unrelated) instructions in between them.

We can also use this technique for other kinds of hazards. Consider program **test9c.txt** and its resource dependency graph as given below:-

Initial Program		**Reordered Program**
i0. LDI R5,12 i1. LDI R6,10 i2. IDIV R7,R8,R1 i3. IDIV R8,R7,R2 i4. STR R5,R1 i5. STR R6,R2		**Reordered Program** i1. LDI R5,12 i3. IDIV R7,R8,R1 i5. STR R5,R1 i2. LDI R6,10 i4. IDIV R8,R7,R2 i6. STR R6,R2
	(a) (b) (c)	

We can see in the original code that there are various true data dependencies between the store instructions and their respective operands. Diagram **(a)** shows these dependencies as a register dependency graph with blue arrows.

However **i2** and **i3** both use an IALU for an IDIV operation, which means that the IALU might[3] be a cause of a structural hazard. Run **week9.cpp** on **test9c.txt** and you will indeed see that the second IDIV is forced to wait for several clock cycles because the first IDIV is keeping the ALU busy.

Diagram **(b)** shows the register dependencies (in blue) along with a new hazard dependency (in red) between i3 and i4. This new dependency represents a structural conflict, in other words, a potential hazard formed by two instructions wanting to use the same single IALU resource.

You will note that this case has been dealt with by our tools by inserting stalls, but that wastes clock cycles as we have observed. We could of instead increase the IALU count, but this introduces additional hardware costs, so is not a cost-free solution either.

Are there other alternatives ? Can we use reordering here instead ?

The answer in this case is yes: If we now treat this new arrow as a dependency in the same way as before, we will want to separate those two instructions in time to try and reduce or remove the structural hazard. However those two instructions can only move in a way that maintains their order as far as the (blue) register dependencies are concerned.

There are permutations that satisfy those constraints. We can see in diagram **(c)** that this can be achieved by reordering the instructions as shown, and as presented in **test9d.txt.** Run **week9.cpp** on **test9d.txt** and you will see that it uses less clock cycles than **test9c.txt**.

In other words, the new instruction ordering allows the CPU to execute the code in less clock cycles whilst completing the same work. We have optimised the code using instruction reordering to eliminate a structural hazard between two ALU operations. We did not waste cycles using stalls and we did not need additional hardware in the CPU.

[3] might - as always we cant be sure unless we plot a pipeline diagram to examine the timing, static scheduling either makes assumptions and tries to avoid potential risks wherever they are found or it has to be sophisticated enough to predict the pipeline behaviour and only identify and fix actual hazards.